Bug Report: Payment Amount Tampering via Client-Side / Untrusted Checkout Parameter

Target URL: https://1000aqi.com/checkout
Reported by: (Reddy Deliph Kumar)
Testing Tool: Burp Suite (intercept + modify requests)

Summary

A tester was able to tamper with the **payable amount displayed on the checkout/payment flow** by intercepting and modifying a request in Burp Suite. The payable total changed from **₹3,999 to ₹1**, indicating **insufficient server-side validation** of the payment amount during the checkout/payment initiation process.

Severity

High — Payment integrity / financial impact

Impact

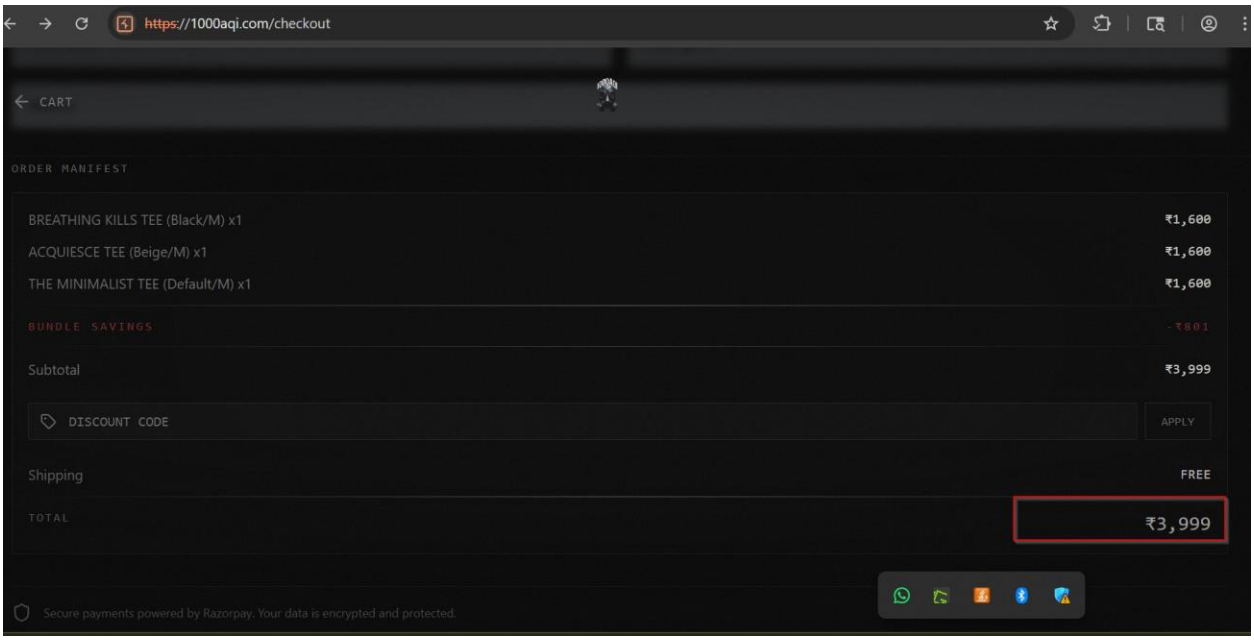
Attackers can:

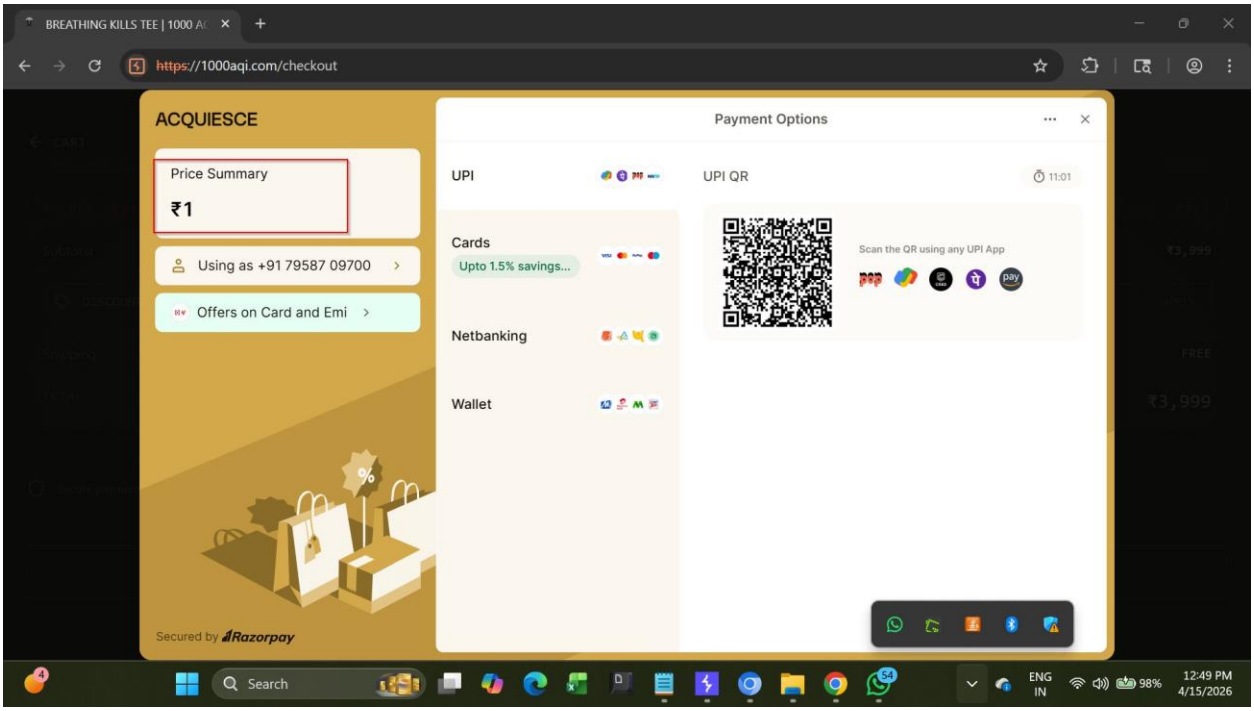
- Manipulate the amount to pay **significantly less than the real cart total**
- Cause incorrect order fulfillment/billing at an attacker-controlled price
- Potentially trigger fraud, revenue loss, disputes, and complex chargeback handling
  Even if the payment gateway performs its own checks, this issue shows the application **does not enforce correctness based on server-trusted calculations**.

Evidence

Screenshot 1 (Before tampering): Total displayed as **₹3,999**

Screenshot 2 (After tampering): Total displayed as **₹1**





Screenshot also shows the payment UI reflecting the tampered amount (₹1 in the payment summary/modal).

Environment

- **Browser:** Chrome
- **Region/Currency:** INR (₹)

- **Endpoint / Page:** Checkout: https://1000aqi.com/checkout
- **Gateway/Modal UI:** Razopay/Razorpay payment flow shown in screenshots

---

Steps to Reproduce

1. Open: https://1000aqi.com/checkout
2. Observe the cart/checkout total (**expected:** ₹3,999).
3. Start **Burp Suite** interception.
4. Identify the request involved in initiating payment / creating the payment intent/order (the step where the amount is sent).
5. Modify the parameter controlling the amount/total (or the amount sent to backend/payment creation).
6. Forward the request to the server.
7. Return to the browser/payment modal.
8. Observe the payable amount has changed to **₹1**.

Expected Result

- The checkout system **must compute payable amount server-side** from trusted sources (cart items, pricing rules, coupons).
- Any client-sent amount/total should be **ignored**.
- Payment creation must be tied to a **server-generated immutable order** with a server-calculated amount.

Actual Result

- The payable amount displayed in the checkout/payment flow was successfully changed from **₹3,999 to ₹1** after request tampering.
- UI/payment flow reflected the tampered value, implying **insufficient server-side enforcement**.

Likely Root Cause

The application appears to rely on **untrusted frontend/client-controlled values** (or unvalidated request parameters) when determining payment amount during the checkout/payment initiation flow.

Common causes:

- Backend accepts a client-provided amount/total without verifying it matches the server-calculated cart/order total
- Payment intent/order is created using an amount derived from the request rather than from the database/cart canonical pricing

---

Recommended Fix (Security Requirements)

1) Server-Side Amount Validation (Mandatory)

On every payment initiation request:

- Recalculate the order total on the backend using:
  - cart contents
  - server pricing
  - coupons/discount rules
- Compare it to any incoming “amount/total” from the client.
- **If mismatch → reject request + log security event**.

2) Do Not Accept Client-Controlled Amount

- Remove/ignore any frontend fields like amount, total, cartTotal, etc.
- Payment creation should use **only server-derived** amount.

3) Bind Payment to a Server-Generated Order

- Create a backend order first:
  - order_id
  - amount calculated server-side
- The payment flow should reference that order_id and not allow overriding amount.

4) Verify Amount on Gateway Webhook/Callback

When receiving Razorpay webhooks/callbacks:

- Verify:
  - order_id
  - amount
  - currency
  - authenticity/signature
- Mark order as paid **only after** webhook verification confirms the expected amount.

5) Monitoring & Logging

- Log suspicious attempts:
  - amount mismatch
  - unusual order creation parameters
  - repeated tampering patterns
- Add alerting for high-risk deviations.